

Exercise 1 — Electronic Device Management

Build a Java console application to manage electronic devices sold in a store, using an in-memory `ArrayList`. The menu runs continuously until the user chooses to exit.

Abstract class `ElectronicDevice`

- Common fields: `deviceId`, `deviceName`, `manufacturer`, `sellingPrice`, `importDate` (`LocalDate`, entered/displayed as yyyy-MM-dd).
- Abstract method `displayInfo()`, overridden by every subclass — the override must include the import date, not just the type-specific fields.

Static nested class `DeviceStatistics`

- Tracks: total device count, Phone count, Laptop count, and the average selling price across all devices.
- Keep this as static counters that update automatically the moment a Phone or Laptop is constructed (e.g. incremented inside each subclass's constructor, or via a shared static method both constructors call) — not recomputed from scratch on demand. Nothing in the menu ever removes a device, so counters only ever need to go up.

Phone (extends `ElectronicDevice`)

- Adds `batteryCapacity` (must be > 0) and a boolean 5G-support flag.
- Overrides `displayInfo()`. Implements `Discountable` with a 10% discount.

Laptop (extends `ElectronicDevice`)

- Adds `ram` (must be > 0) and `screenSize`.
- Overrides `displayInfo()`. Implements `Discountable` with a 15% discount.

Discountable interface

- `double calculateDiscountPrice();` — Phone returns `sellingPrice * 0.90`, Laptop returns `sellingPrice * 0.85`.

Custom exceptions

- `DuplicateDeviceIdException` — the entered device ID already exists.
- `InvalidPriceException` — selling price is not > 0.
- `InvalidAttributeException` — the original spec describes this exception (triggered when battery capacity or RAM isn't > 0) but never actually names it; use this name, or any name you prefer, consistently.
- `DeviceNotFoundException` — declared for when no device matches a requested ID, but note that none of the 15 menu functions below actually need to throw it: the one ID-based search (function 2) explicitly returns an empty `Optional` instead of throwing. It's fine to leave this exception defined but unused, since nothing in this menu calls for an update-or-delete-by-ID operation.
- Also catch `InputMismatchException` (from `Scanner`) wherever a numeric field is read from the keyboard, so entering text where a number is expected doesn't crash the program — re-prompt instead.

Menu functions

- 1 — Add a device (Phone or Laptop): validate ID uniqueness (`DuplicateDeviceIdException`), price > 0 (`InvalidPriceException`), the type-specific attribute (`InvalidAttributeException`), and collect the import date as yyyy-MM-dd.
- 2 — Search by ID: returns `Optional<ElectronicDevice>` via the Stream API; display the device if present, otherwise a "not found" message — no exception on this path.
- 3 — List phones that support 5G.

- 4 — List laptops with RAM $\geq$ 16GB.
- 5 — Find the phone with the largest battery capacity.
- 6 — Find the laptop with the largest screen size.
- 7 — Find the device with the highest selling price, via Stream + Optional; if the list is empty, show a message instead of an error.
- 8 — Compute the total value of all devices.
- 9 — Find devices by manufacturer (user enters the name).
- 10 — List device names, built with `.map()`.
- 11 — List devices imported in a year the user specifies.
- 12 — Show the discounted price of every device: ID, name, original price, discounted price.
- 13 — Sort by name (A→Z), 14 — sort by price (ascending): both via a Lambda Expression passed to `Collections.sort()/List.sort()` — no standalone Comparator classes.
- 15 — Show device statistics via DeviceStatistics: total devices, Phone count, Laptop count, average selling price.

General requirements

- Use inheritance for shared attributes/behavior; Phone and Laptop implement Discountable.
- Apply Lambda Expressions and the Stream API to every filter/search/sort/compute operation above, replacing traditional for-loops; use method references in place of lambdas wherever that's a clean fit (e.g. a bare getter reference in a comparator or map()).
- Every search returns Optional, never null.

Exercise 2 — Transportation Management System

Build a Java console application managing a transportation center's vehicles. Car and Motorcycle objects are stored together in one ArrayList, and the menu runs continuously until the user exits.

Abstract class Vehicle

- Common fields: `vehicleId`, `vehicleName`, `manufacturer`, `vehicleValue`, `registrationDate` (LocalDate, entered as yyyy-MM-dd; on a parse failure, catch it and re-prompt with a message like `Invalid date format. Please enter the date using yyyy-MM-dd.`).
- Implements `Comparable<Vehicle>` — natural ordering by vehicle value, ascending. Declares `public abstract double calculateAnnualTax();`, overridden by each subclass.
- Every subclass overrides a display method (e.g. `displayInfo()`) showing its own type-specific fields; call it polymorphically through a Vehicle reference wherever possible, rather than casting to the concrete type first.

Static nested class VehicleStatistics

- Auto-updated whenever a vehicle is added. Exposes: `getTotalVehicles()`, `getTotalCars()`, `getTotalMotorcycles()`, `getTotalVehicleValue()` — all `public static`.

Non-static inner class VehicleInsurance

- Fields: insurance provider, coverage amount. Every `Vehicle` must contain exactly one, entered by the user at creation and shown as part of `displayInfo()`.
- Because it's a *non-static* inner class, a `VehicleInsurance` can't be constructed on its own — it needs an enclosing `Vehicle` instance, with syntax like `someVehicle.new VehicleInsurance(provider, amount)` from outside the class, or just `new VehicleInsurance(provider, amount)` from inside one of Vehicle's own constructors. This is easy to get wrong on a first attempt if you haven't used a non-static inner class before.

Car (extends Vehicle)

- Adds `numberOfSeats` (must be > 0) and `fuelType` (enum: Gasoline / Diesel / Electric).
- `calculateAnnualTax()`: Gasoline → 5% of vehicle value, Diesel → 6%, Electric → 3%.
- `getRegistrationStatus()` (from `Registrable`): Electric → "Green Vehicle"; Gasoline or Diesel → "Standard Vehicle".

Motorcycle (extends Vehicle)

- Adds `engineCapacityCc` (must be > 0) and a boolean ABS-supported flag.
- `calculateAnnualTax()`: under 150cc → 2% of vehicle value, 150cc or more → 4%.
- `getRegistrationStatus()`: ABS → "Safety Certified"; no ABS → "Basic Certified".

Registrable interface

- `String getRegistrationStatus();` — implemented by both Car and Motorcycle.

Custom exceptions

- `DuplicateVehicleIdException` — the entered vehicle ID already exists.
- `InvalidVehicleValueException` — vehicle value ≤ 0, number of seats ≤ 0, or engine capacity ≤ 0 (one shared exception covering all three triggers).
- Handle every exception gracefully with a clear message and let the user re-enter the value; the program must never crash on invalid-type keyboard input either.

Menu functions

- 1 — Add a Car, 2 — Add a Motorcycle: validated against both custom exceptions above, and also collecting registration date and VehicleInsurance info.
- 3 — Display all vehicles.
- 4 — Search by ID, 5 — Search by name: both via the Stream API, returning `Optional<Vehicle>` (never null); show full info if found, else "Vehicle not found."
- 6 — List all electric cars. 7 — List all motorcycles with ABS. 8 — List vehicles by manufacturer.
- 9 — Sort by value ascending, via natural ordering (`Comparable<Vehicle>`). 10 — sort by name (A→Z). 11 — sort by manufacturer (A→Z). 12 — sort by annual tax (highest→lowest). All four sorts use a Lambda Expression passed directly to `Collections.sort()/List.sort()` — no standalone Comparator classes.
- 13 — Tax report: for every vehicle, show ID, name, type, annual tax, and registration status, via polymorphic calls to `calculateAnnualTax()` and `getRegistrationStatus()`.
- 14 — Statistics via `VehicleStatistics`: total vehicles, total cars, total motorcycles, total vehicle value.
- 15 — List vehicles registered within the last N days (user enters N); include a vehicle if its registration date falls within that many days of today.

General requirements

- Vehicle IDs must be unique. Use polymorphism by storing both Car and Motorcycle in one `ArrayList<Vehicle>`; use method references in place of lambdas wherever that's a clean fit.
-

Exercise 3 — Library Document Management (Interface & Abstract Class)

A library manages Books, Magazines, and Newspapers, sharing a Document ID (unique), Publisher Name, and Number of Copies Issued. Books also track Author Name and Number of Pages; Magazines track Issue Number and Issue Month; Newspapers track Issue Date.

Class design

- An interface, e.g. `IDisplayable`, declaring `void display();` — the contract every document type fulfills.
- An abstract class `Document` implementing `IDisplayable`, holding the three shared fields and a constructor for them; `Book`, `Magazine`, and `Newspaper` extend `Document`, each overriding `display()` to print the shared fields plus its own extra ones.

DocumentManager class

- Add a new document (Book, Magazine, or Newspaper).
- Remove a document by Document ID.
- Display information about a document — this means the full stored collection (every document currently held, each printed via its own `display()`), as distinct from the type-filtered search below.
- Search for documents by type (Book, Magazine, or Newspaper) and display the matches.
- Exit the program.

Exercise 4 — University Admissions (Interface & Abstract Class)

Candidates sit for exam group A, B, or C. Every candidate has a Candidate Number, Full Name, Address, and Priority Level. Group A takes Math/Physics/Chemistry; Group B takes Math/Chemistry/Biology; Group C takes Literature/History/Geography.

Class design

- An abstract class `Candidate` holding the four shared fields (Priority Level as a plain int — the exercise doesn't ask for any bonus-point calculation from it, so it just needs to be stored and displayed).
- Three subclasses, one per group, each adding three `double` score fields named for its own subjects (e.g. `GroupACandidate` has `mathScore`, `physicsScore`, `chemistryScore`). Inheritance alone — an abstract method on `Candidate` that each subclass overrides to total its own three scores — is enough here; a separate interface isn't needed on top of that unless you want one.

AdmissionManager class

- Add a new candidate.
- Display a candidate's information along with their exam group.
- Search for a candidate by Candidate Number.
- Exit the program.

Exercise 5 — Document Management (Interface)

IDocument interface

- `void input();` and `void display();`

Book class

- Private fields: `id` (int), `bookTitle` (String), `authorName` (String), `price` (float). Implements `IDocument`.
- A no-argument constructor and a 4-argument constructor; getters/setters for all four fields.
- `input()`: prompts for and sets all four fields from the keyboard.
- `display()`: prints all four fields.

DocumentManager class

- Keep the book collection as a field on DocumentManager that persists across calls — if you use a plain array, that means growing it (e.g. into a new, larger array copied from the old one) each time `addDocument` runs, not replacing it outright. Otherwise, calling "Add new book" a second time would silently erase every book entered the first time.
- `addDocument()`: asks for the number of books `n` to add *this call*, then collects `n` books' information, extending the persistent collection by those `n` books.
- `displayAllDocuments()`: prints every book currently stored, via each one's `display()`.
- `searchByAuthor(String authorName)`: prompts for an author name (or takes it as given, per the signature), then prints every book by that author, or a "not found" message if there are none.

DocumentTest class (with main method) and menu:

-
1. Add new book
 2. Display all books
 3. Search books by author
 4. Exit
-

Menu behaviour

- 1 — call `addDocument()`, then return to the menu.
 - 2 — call `displayAllDocuments()`, then return to the menu.
 - 3 — prompt for an author's name, call `searchByAuthor()` with it, then return to the menu.
 - 4 — end the program.
-

Exercise 6 — Outdoor Trip Management System

Abstract class Trip

- Protected fields: `id` (String), `destination` (String), `date` (String).
- `numberOfParticipants`: the original spec types this as a String, but menu option 3 below needs to sort trips by this value numerically (descending). String comparison is lexicographic, not numeric — e.g. the text "9" sorts *after* the text "10", which is backwards for a participant count. Declare this field as an int instead; that's the one field in this exercise where following the original's stated type would produce a genuinely wrong result, not just an inconvenient one.
- A default constructor and a parameterized constructor; getters/setters for all fields; two abstract methods, `public abstract void enterInfo();` and `public abstract void showInfo();`

FamilyTrip (extends Trip)

- Adds `hasChildren` (private boolean) and `familyType` (private boolean) — the original doesn't say what true/false means for `familyType`; treat it as "traveling with extended family / relatives" (true) vs. "immediate family only" (false), and keep that meaning consistent everywhere you use it.
- A no-argument constructor and a 6-argument constructor (4 inherited fields + these 2), using `super(...)` to set the inherited fields.

- Getters/setters for the two new fields.
- `enterInfo()`: prompts for every field, inherited and new. `showInfo()`: prints every field.

AdventureTrip (extends Trip)

- Adds `activityType` (private String) and `difficultyLevel` — the original doesn't state a type for this one either, but it's multiplied by a per-day rate and sorted numerically (menu option 4), so it must be numeric: declare it as an int.
- A no-argument constructor and a 6-argument constructor, using `super(...)` for the 4 inherited fields.
- Getters/setters for the two new fields. `enterInfo()` / `showInfo()` following the same pattern as FamilyTrip.

estimateDuration()

- New method on `AdventureTrip`: Hiking → 1 day per difficulty level; Rafting → 1.5 days per level; Mountain Climbing → 2 days per level.
- Match `activityType` case-insensitively against these three names. If it doesn't match any of them, return 0 and note "Unknown activity type" rather than silently computing a meaningless number.
- After `showInfo()` runs (as part of menu option 4 below), also print the estimated duration for that trip. Expected result for a Hiking trip at difficulty 4:

```
Trip ID: TR001
Destination: Grand Canyon
Activity Type: Hiking
Difficulty Level: 4
Estimated Trip Duration: 4 days
```

TripTest class (with main method) and menu:

```
Please select:
1. Enter information for n family trips.
2. Enter information for n adventure trips.
3. Display information for n family trips (sorted by number of participants, descending).
4. Display information for n adventure trips (sorted by difficulty level, ascending).
5. Exit.
Your choice:
```

Menu behaviour

- 1 — ask how many family trips to enter, then collect that many into a FamilyTrip array.
 - 2 — ask how many adventure trips to enter, then collect that many into an AdventureTrip array.
 - 3 — display the FamilyTrip array sorted by number of participants, descending.
 - 4 — display the AdventureTrip array sorted by difficulty level, ascending (each entry followed by its estimated duration, per the example above).
 - 5 — exit.
-

Exercise 7 — Hotel Management (Abstract Class & Interface)

Manages guests renting hotel rooms. Room type A costs \$500/night, type B \$300/night, type C \$100/night.

PersonAbs / Person

- Abstract class `PersonAbs`: fields `name`, `age`; abstract method `goToWork()` (no body — abstract methods can't have one).

- Class `Person` extends `PersonAbs`, adding `idNumber`. Since `goToWork()` is abstract in the parent, `Person` must provide the actual implementation — printing exactly "go to work".

IHotel interface and Hotel class

- Interface `IHotel` with three methods: something like `void addCustomer(Person guest, String roomType, int nights);`, `void deleteCustomer(String idNumber);`, and `double calculatePrice(String idNumber);` (exact signatures aren't given in the original, so these are a reasonable choice — adjust names/parameters if you prefer, as long as the three operations are covered).
- Class `Hotel` implements `IHotel`. For each guest it needs to remember which room type they chose and how many nights, in addition to their `Person` info (room type and nights are per-stay, not permanent facts about the person) — a small internal record combining the three is enough; it doesn't need to be its own top-level class.
- Reject a room type outside A/B/C and re-prompt.

Add / remove / calculate

- Add a new guest (with their room type and nights). Remove a guest by ID number.
- Calculate a guest's room charge (looked up by ID number): `nights × price-per-night-of-their-room-type` (\$500 / \$300 / \$100 for A / B / C).

Exercise 8 — Transportation Management (Interface)

IVehicle interface

- `void startEngine();`, `void stopEngine();`, `void brake();`

Vehicle class (implements IVehicle)

- Common fields: `id`, `manufacturer`, `yearOfManufacture`, `sellingPrice`, `color`.
- `stopEngine()` and `brake()` can share one generic implementation here (e.g. print "Stopping engine" / "Braking") inherited as-is by all three subtypes — only `startEngine()` needs type-specific behavior below.

Car, Motorcycle, Truck (extend Vehicle)

- Car adds `numberOfSeats` (Integer) and `engineType` (String).
- Motorcycle adds `enginePower` (Integer).
- Truck adds `loadCapacity` (Integer).
- Each overrides `startEngine()` with its own message: Motorcycle prints "Starting motorcycle engine" (given explicitly); for consistency, Car should print "Starting car engine" and Truck "Starting truck engine" (the original gives only Motorcycle's exact wording and says the other two are "similar" without spelling them out).

Truck.loadCargo(Integer weight)

- If weight exceeds `loadCapacity`, print "Load weight is not suitable"; otherwise print "Load weight is suitable".
- Return value: the original declares this method returns an Integer but doesn't say what it means. For symmetry with `boardPassengers` below, return the excess over capacity (`weight - loadCapacity`) when it's too heavy, or 0 when it fits.

Car.boardPassengers(Integer numberOfPassengers)

- If `numberOfPassengers` exceeds `numberOfSeats`, print a message noting how many passengers don't fit; otherwise print a message noting everyone fits.
- Return the number of passengers exceeding capacity (0 if they all fit) — this is given explicitly in the original.

VehicleManager class

- Add/remove (by ID) any of the three vehicle types, held together in one `List<Vehicle>`.
- Call `loadCargo` on a truck by ID — if the ID belongs to a Car or Motorcycle instead, report that it isn't a truck rather than crashing with a cast exception.
- Call `boardPassengers` on a car by ID — same not-the-right-type check as above.
- Search vehicles by manufacturer or by color.
- Exit.

Exercise 9 — Warehouse Management (Interface & Inheritance)

IProduct interface and Product class

- `IProduct`: `void addStock(int quantity); void removeStock(int quantity); void displayInfo();`
- `Product` implements `IProduct` with fields `id`, `name`, `price`, `quantity`. `addStock()` increases `quantity` by the given amount; `removeStock()` decreases it — but if the requested amount exceeds current stock, reject the whole operation and print an error rather than letting quantity go negative or silently clamping it (a silent clamp would misrepresent how much was actually removed).

Food, Electronics, Household (extend Product)

- Food adds `expirationDate` (String), `isPerishable` (boolean), and `updateExpirationDate(String)`.
- Electronics adds `warrantyPeriod` (String), `brand` (String), and `extendWarranty(String additionalPeriod)` — since both fields are free-text strings rather than numbers, this doesn't need real arithmetic; updating `warrantyPeriod` to reflect the extension (e.g. replacing it with a combined description) is enough.
- Household adds `material` (String), `usage` (String), and `updateUsage(String)`.
- Each of the three overrides `displayInfo()` to show the shared Product fields plus its own — a Food display includes expiration/perishability, etc.

InventoryManager class

- Store products in a `HashMap<String, Product>` keyed by ID, for efficient add/remove/lookup.
- Add a product; remove one by ID.
- Find products by name, and separately, filter by category (food / electronics / household).
- Call `updateExpirationDate` on a Food item by ID, `extendWarranty` on an Electronics item by ID, and `updateUsage` on a Household item by ID — in each case, if the ID found belongs to the wrong category, report that rather than crashing on a bad cast.
- Display all products.

Main class

- An interactive menu covering add / remove / search / display, wired to the InventoryManager methods above.

Exercise 10 — Student Management (Interface)

IStudent interface

- `void input();` and `void display();`

Student class

- Fields: `id` (int), `fullname` (String), `email` (String), `mark` (float). Implements `IStudent`.
- A no-argument constructor and a 4-argument constructor; getters/setters for all four fields.
- `input()`: prompts for and sets all four fields. `display()`: prints all four.

StudentManagement class

- `addStudent()`: collects exactly 3 `Student` objects per call, as specified. Keep the underlying array as a persistent field that grows by 3 on each call, rather than being replaced — otherwise calling "Add New" a second time would silently erase the students entered the first time.
- `showStudent()`: iterates and displays every stored student.
- `public Student sortStudentByMark()`: sorts the stored array by mark, and returns the top-ranked student (highest mark) after sorting — the declared return type is a single `Student`, so this gives it a concrete, useful meaning; have `MainClass` print the returned student so the sort has some visible result beyond re-ordering an array no one sees.

`MainClass` (with main method) and menu:

```

=== MENU ===
1. Add New.
2. Show All.
3. Sort.
4. Exit.
Your choice:

```

Menu behaviour

- 1 — run `addStudent()`. 2 — run `showStudent()`. 3 — run `sortStudentByMark()` and print the student it returns. 4 — exit.

Exercise 11 — Book Library Management (Interface & Inheritance)

IBook interface and Book class

- `IBook`: `void addCopies(int quantity); void removeCopies(int quantity); void displayInfo();`
- `Book` implements `IBook` with fields `id`, `title`, `author`, `price`, `quantity`. `addCopies()/removeCopies()` adjust `quantity` the same way `addStock/removeStock` do in a warehouse system — reject a removal that would take quantity below 0 rather than clamping or going negative.

Textbook, Novel, ReferenceBook (extend Book)

- `Textbook` adds `subject` (String), `academicLevel` (String), and `updateAcademicLevel(String)`.
- `Novel` adds `genre` (String), `isBestseller` (boolean), and `markAsBestseller()` (no parameter — just sets `isBestseller = true`).
- `ReferenceBook` adds `fieldOfStudy` (String), `edition` (String), and `updateEdition(String)`.
- Each overrides `displayInfo()` to include the shared `Book` fields plus its own.

LibraryManager class

- Store books in a `HashMap<String, Book>` keyed by ID, for the same reason as the warehouse exercise.
- Add a book; remove one by ID.
- Find books by title, and separately, by category (textbook / novel / reference).

- Call `updateAcademicLevel` on a Textbook by ID, `markAsBestseller` on a Novel by ID, and `updateEdition` on a ReferenceBook by ID — report a wrong-category ID rather than crashing on a bad cast, same as the warehouse exercise.
- Display all books.

Exercise 12 — Hospital Doctor Management (Abstract Class)

Abstract class Doctor

- Protected fields: `doctorId` (String), `doctorName` (String), `specialization` (String), `salary` (float).
- A default constructor and a parameterized constructor; getters/setters for all fields; abstract methods `public abstract void input();` and `public abstract void display();`

SpecialistDoctor and Surgeon (extend Doctor)

- SpecialistDoctor adds `expertise` (private String), `yearsOfExperience` (private int).
- Surgeon adds `numberOfSurgeries` (private int), `hospitalName` (private String).
- Each gets a no-argument constructor and a 6-argument constructor (4 inherited fields + its own 2). The original says to use `this` to call the parent constructor — that's not actually possible in Java: `this(...)` calls another constructor in the *same* class, and only `super(...)` can call a parent class's constructor. Use `super(doctorId, doctorName, specialization, salary)` as the first line of the 6-argument constructor, then set the two new fields directly afterward.
- Getters/setters for the new fields; implement `input()` (prompts for every field, inherited and new) and `display()` (prints every field) on both classes.

DoctorTest class (with main method) and menu:

```
Please select:
1. Enter information for n specialist doctors.
2. Enter information for n surgeons.
3. Display information for n specialist doctors (sorted by salary, descending).
4. Display information for n surgeons (sorted by number of surgeries, ascending).
5. Exit.
Your choice:
```

Menu behaviour

- 1 — collect n specialist doctors into a SpecialistDoctor array.
- 2 — collect n surgeons into a Surgeon array.
- 3 — display specialist doctors sorted by salary, descending.
- 4 — display surgeons sorted by number of surgeries, ascending.
- 5 — exit.

Exercise 13 — Computer & Book Product Catalog (Abstract Class)

Abstract class Product

- Protected fields: `proId` (String), `proName` (String), `year` (int), `price` (float).
- Two constructors, getters/setters, and two abstract methods: `public abstract void input();` and `public abstract void display();`

Computer and Book (extend Product)

- Computer adds `speed` (private String), `producer` (private String).
- Book adds `type` (private String), `publisher` (private String).
- Each gets a no-argument constructor and a 6-argument constructor. As in the previous exercise, the original's instruction to use `this` to reach the parent's constructor isn't valid Java — use `super(proId, proName, year, price)` as the first line, then set the two new fields.
- Getters/setters for the new fields; implement `input()` and `display()` on both classes, covering inherited and new fields.

Test class (with main method) and menu:

```
Please select:
1. Input information for n Computers.
2. Input information for n Books.
3. Display information of n Computers by sorting the price descending.
4. Display information of n Books by sorting the publisher ascending.
5. Exit.
Your choice:
```

Menu behaviour

- 1 — collect n computers into a Computer array.
 - 2 — collect n books into a Book array.
 - 3 — display the Computer array sorted by price, descending.
 - 4 — display the Book array sorted by publisher, ascending.
 - 5 — exit.
-

Exercise 14 — Exception Handling Practice

Eight independent, short programs — each one just needs to trigger and handle the named exception with try-catch-finally, not build a larger application.

1. Wrong input type

- Prompt for a number with `Scanner.nextInt()` (or similar); entering non-numeric text throws `InputMismatchException`. Catch it and print a clear message instead of a stack trace.

2. Divide by zero

- Use `int` division for this one specifically: `5 / 0` throws `ArithmeticException` (" / by zero"). `double` division by zero does **not** throw — `5.0 / 0.0` silently produces `Infinity` — so if you use doubles here there will be nothing to catch and the exercise won't demonstrate anything.

3. Opening a file that doesn't exist

- Open a nonexistent path with `FileReader` or `FileInputStream`; this throws `FileNotFoundException`. Catch and print a message.

4. Invalid array index

- Access an array outside its bounds, e.g. `arr[arr.length]`; this throws `ArrayIndexOutOfBoundsException`.

5. Non-numeric string to number

- `Integer.parseInt("abc")` throws `NumberFormatException`. Catch it and print a message.

6. A method can't find a file

- As written, this describes the same scenario as #3. To make it a genuinely separate exercise, use a different API here: `java.nio.file.Files.newBufferedReader(path)` against a missing file throws `NoSuchFileException` rather than `FileNotFoundException` — catch that instead, so #3 and #6 end up practicing two different file APIs rather than duplicating each other.

7. try-catch-finally with a throws declaration

- Write a method declared `throws ArithmeticException` (or a broader `Exception`) whose body performs `int` division that can divide by zero (same reasoning as #2 — `int`, not `double`). Call it from a `try` block, catch the exception, and use a `finally` block to print something that always runs regardless of whether the division succeeded.

8. assert for a positive number

- Use `assert value > 0 : "value must be positive";` on a value read from the keyboard. Java disables assertions by default — an `assert` statement silently does nothing at runtime unless you run the program with the `-ea` (enable assertions) JVM flag. Without that flag, this exercise will appear to do nothing even when the assertion should fail, which isn't a bug in your code.
-